

EECS 482

Introduction to operating systems

Crash consistency

Peter Chen
University of Michigan

Multiple updates and reliability

What disk I/Os are needed to create a new file?

- Option 1:
 1. Update directory to point to new file header
 2. Write new file header

What happens if crash occurs between #1 and #2?
- Option 2:
 1. Write new file header
 2. Update directory to point to new file header

What happens if crash occurs between #1 and #2?

Does the file exist at this point?

Is careful ordering enough?

- Example: Create new file and update free block list
- Option 1:

1. Write new file header
2. Update directory to point to new file header
3. Write the new free block list

What happens if crash occurs between #2 and #3?

- Option 2:

1. Write new file header
2. Write the new free block list
3. Update directory to point to new file header

What happens if crash occurs between #2 and #3?

Is careful ordering enough?

- Transfer \$100 from Peter's account to Janet's account
- Option 1:
 1. Deduct \$100 from Peter's account
 2. Add \$100 to Janet's account

What happens if crash occurs between #1 and #2?

- Option 2:
 1. Add \$100 to Janet's account
 2. Deduct \$100 from Peter's account

What happens if crash occurs between #1 and #2?

What does this remind you of?

Transactions

- A set of persistent updates that appear to be **atomic** and **durable**, even if the system crashes
 - ACID: **Atomic**, Consistent, Isolated, **Durable**

```
begin transaction
  write disk
  write disk
  write disk
end (commit) transaction
```

Atomicity and durability

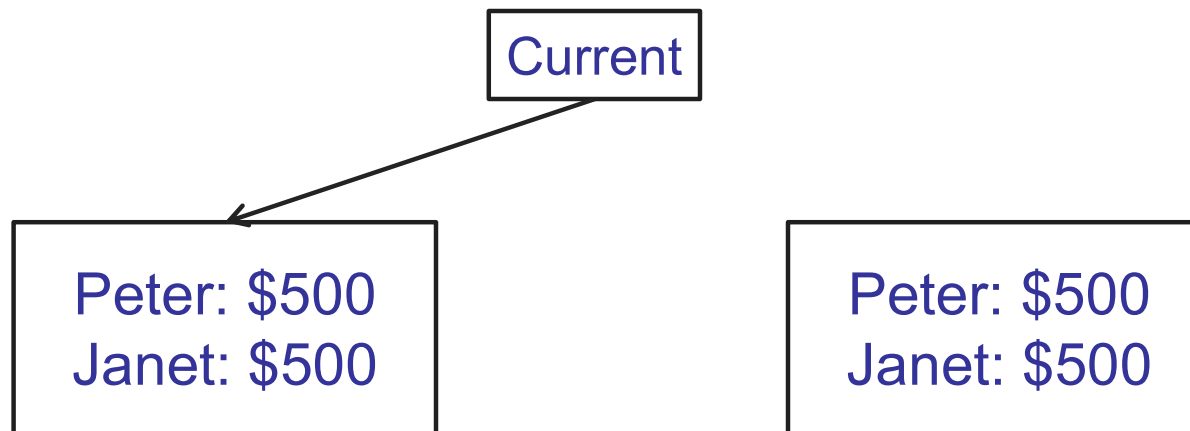
- Atomic (with respect to crashes): only **two** acceptable outcomes (no matter if/when crash occurs)
 - All disk writes in the transaction complete
 - No disk writes in the transaction occur
- Durable: after transaction commits, only **one** acceptable outcome
 - All disk writes in the transaction complete
- What does it mean for a disk write to complete?

Implementing transactions

- How did we make a critical section appear atomic with respect to other threads?
- Building block for transactions: writing a single disk sector is atomic (or pretty close)
 - Sector has the new data (“all”)
 - Section has the old data (“nothing”)
 - ~~Sector is detectably corrupted~~
- How to make a set of disk writes atomic?
 - Shadowing
 - Logging

Transactions via shadowing

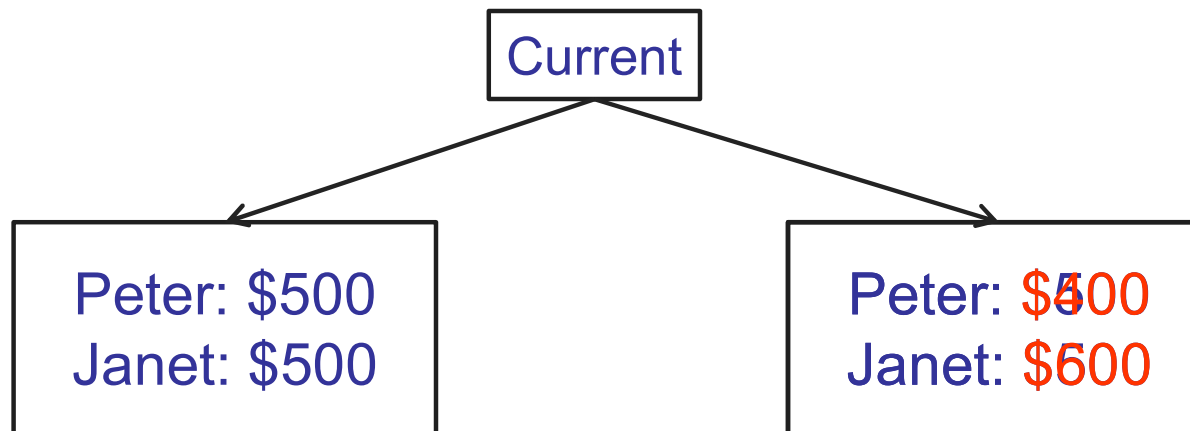
- Keep two copies of the data: current version + backup (shadow)
 - Keep a pointer (also on disk) to current version



- At beginning of transaction, both copies are identical

Shadowing

- During transaction, write all new data to the backup (shadow)
- At end of transaction, switch the pointer to point to the backup

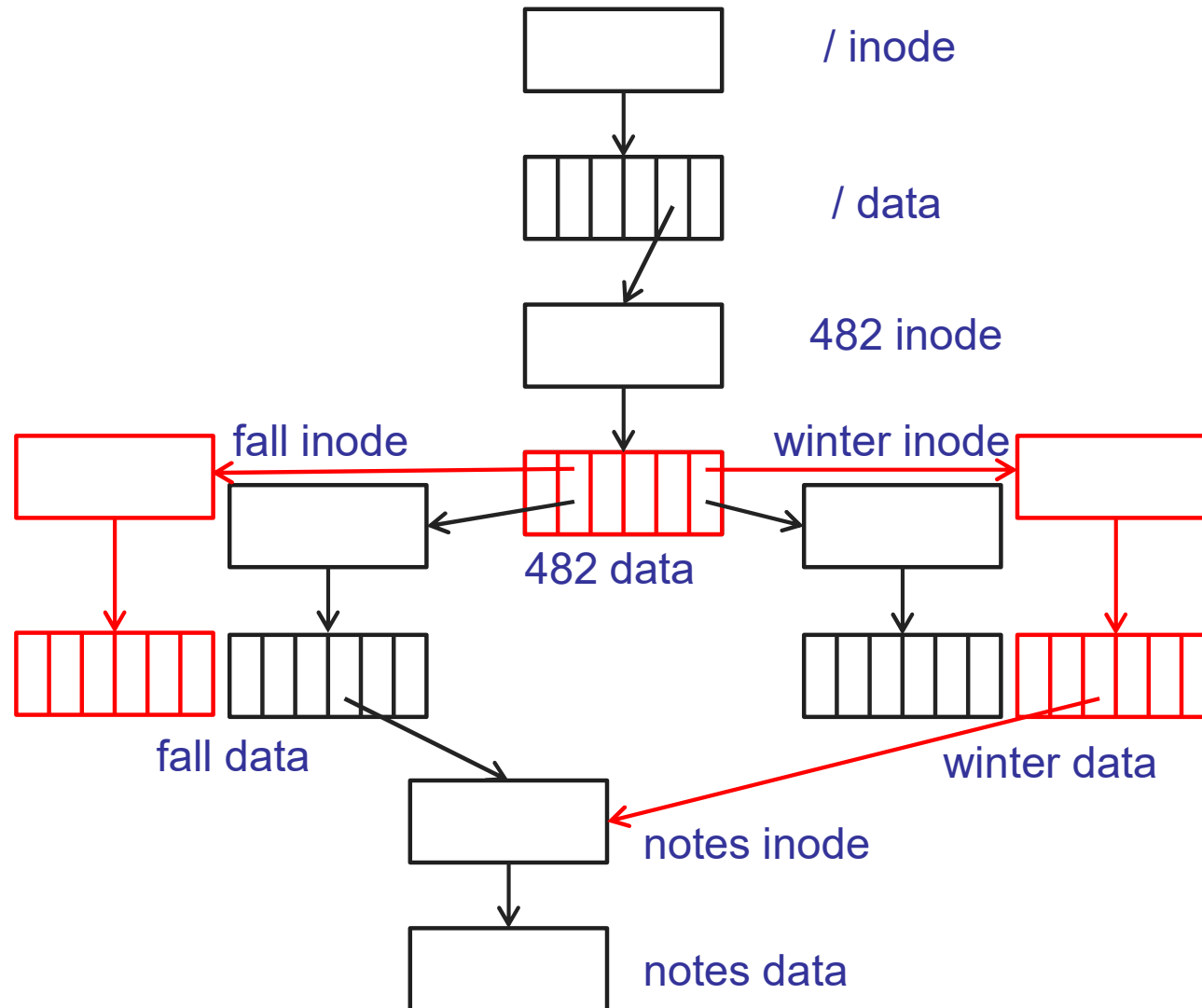


- A single-sector disk write commits multiple disk writes
 - Indirection to the rescue (again)!

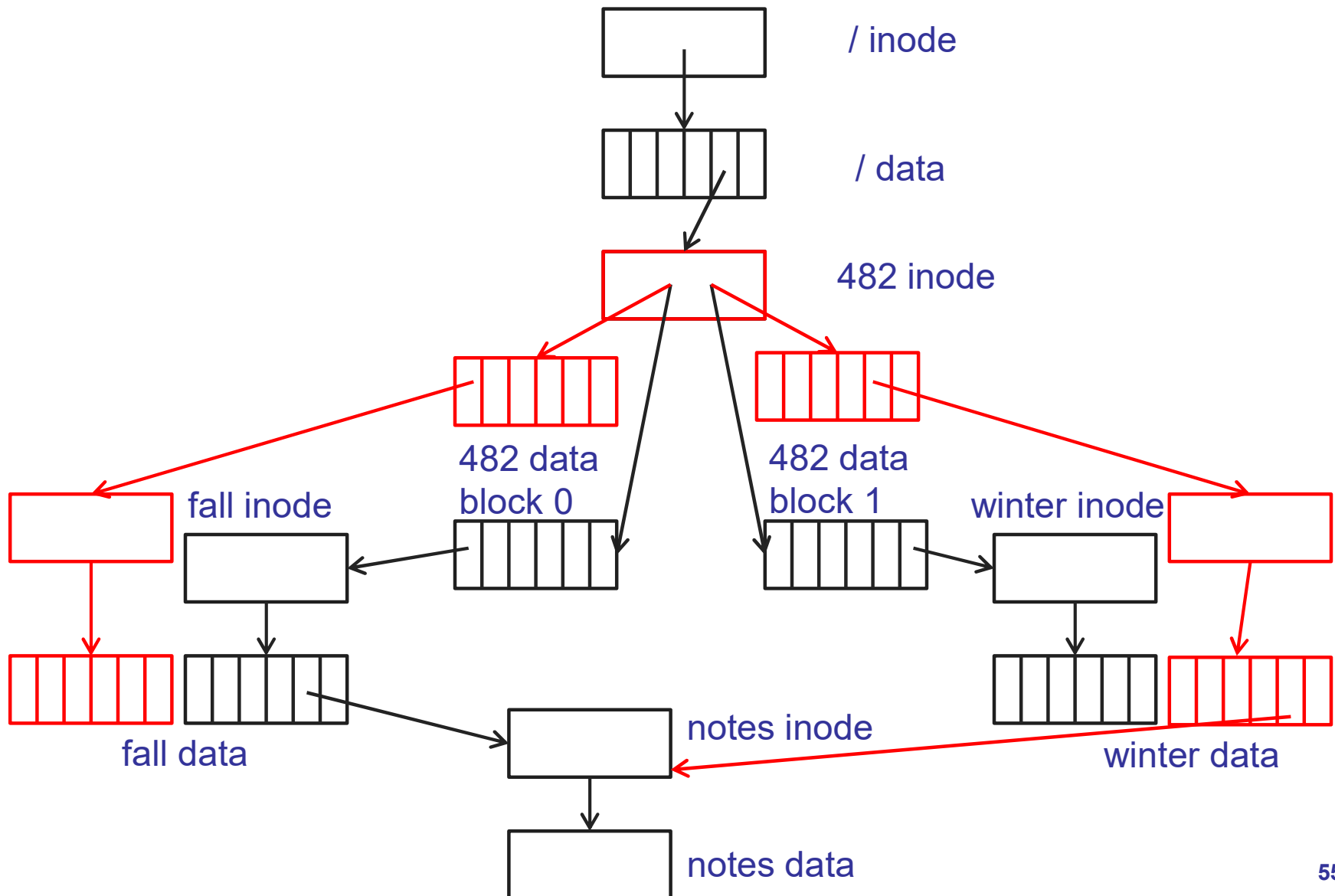
Shadowing summary

- Makes an arbitrary set of updates atomic
- Problems?
- Reduce cost by shadowing on demand
 - Defer and avoid work!
- Leverage the ability to store more than a single pointer in a disk sector

Example: move notes from /482/fall/ to /482/winter/



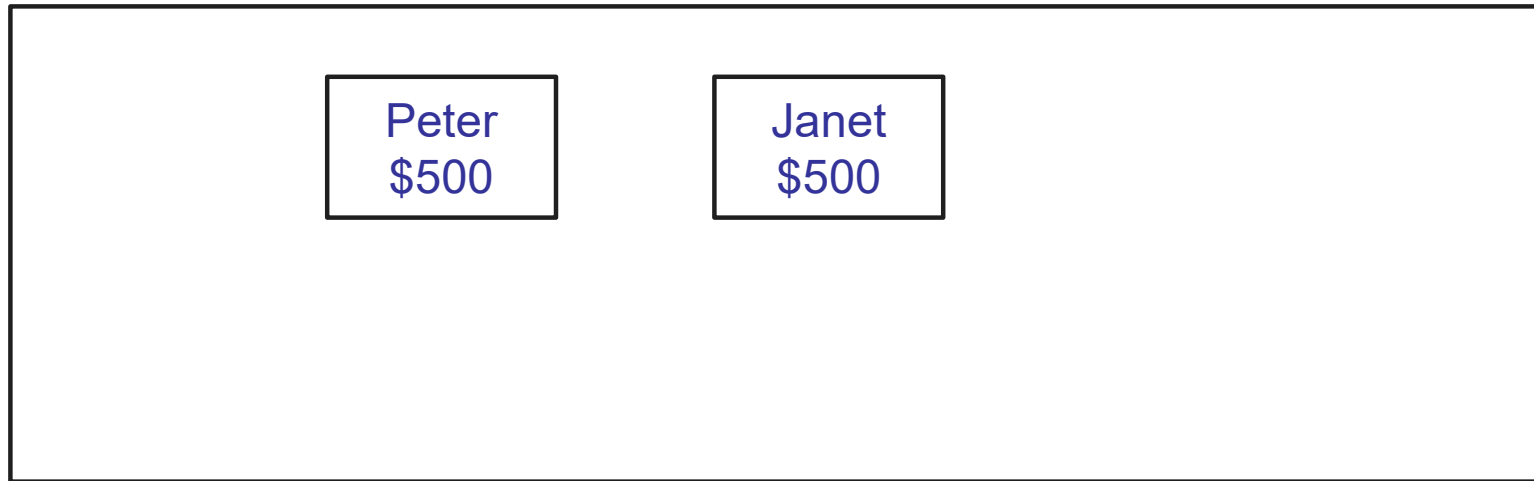
Example: move notes from /482/fall/ to /482/winter/



Transactions via logging

- Divide storage into:
 - **Data store:** Persistent copy of data
 - **Log:** Append-only record of changes

Data
store



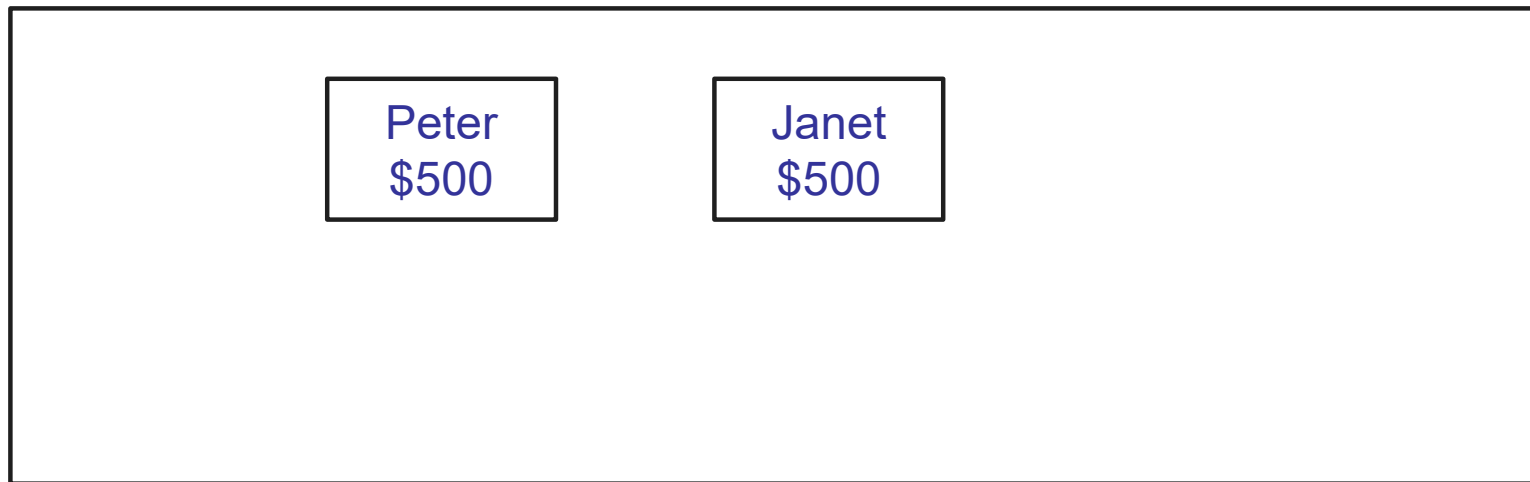
Log



Logging

- Step 1: Append updates to log
 - Data store not updated
 - On crash, discard log (“nothing”)

Data
store



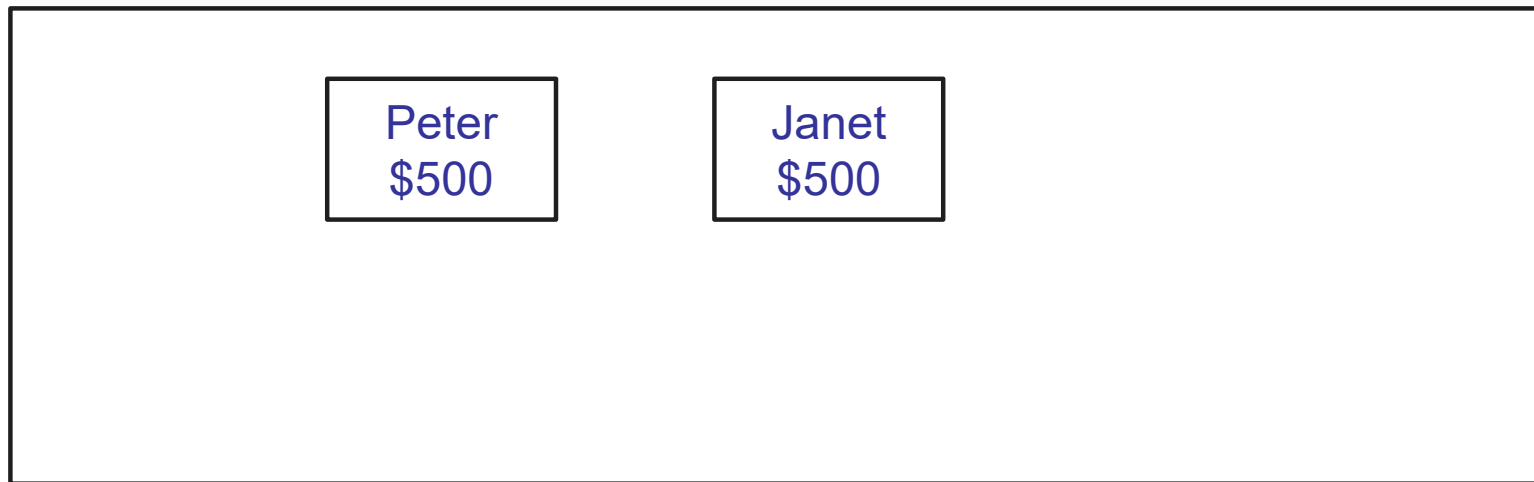
Log



Logging

- Step 2: Commit transaction
 - Append “commit” record to log (“nothing” -> “all”)

Data
store



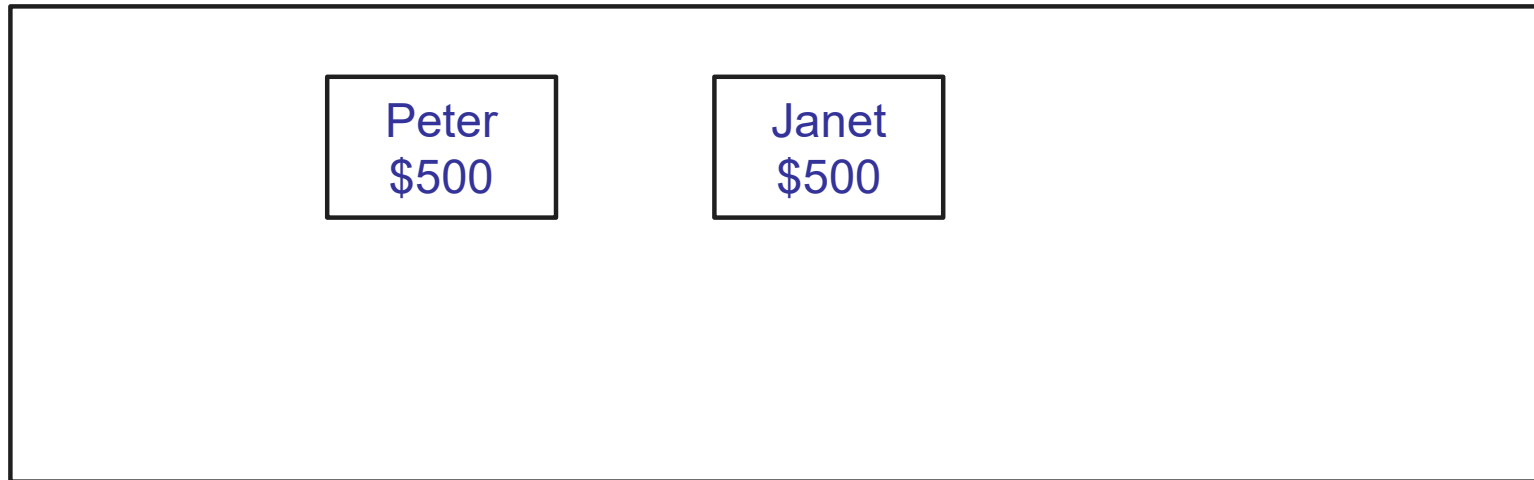
Log

Peter \$400	Janet \$600	Commit	
----------------	----------------	--------	--

Logging

- Step 3: Update data store
 - Copy updates from log to data store
 - Clear log
- Problem?

Data
store



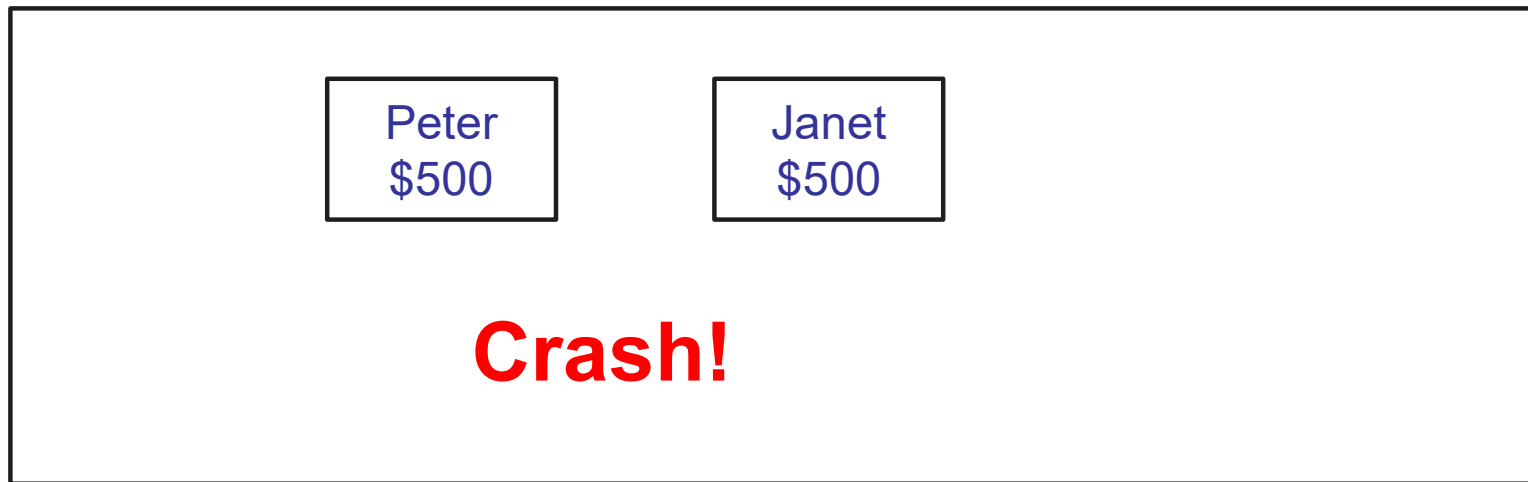
Log



Logging

- Step 3: Update data store
 - Copy updates from log to data store
 - Clear log
- Problem?

Data store



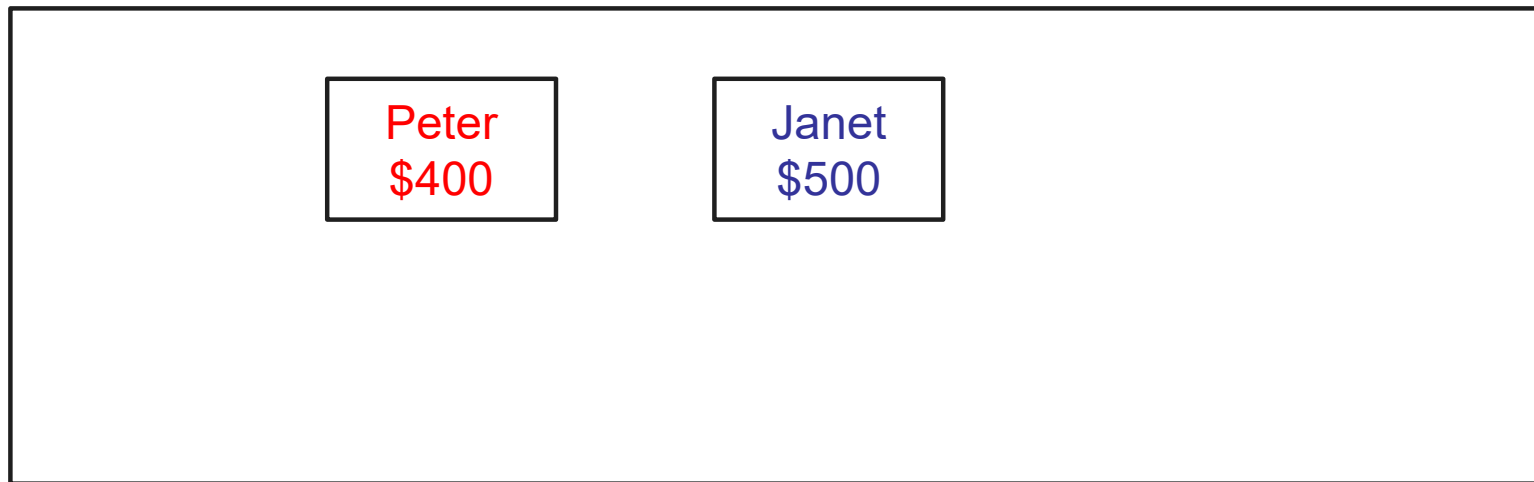
Log

Peter \$400	Janet \$600	Commit	
-------------	-------------	--------	--

Logging

- Step 4: If we crash before applying all updates, **recover** by re-applying all updates in log until last commit record, then clear log.

Data
store



Log

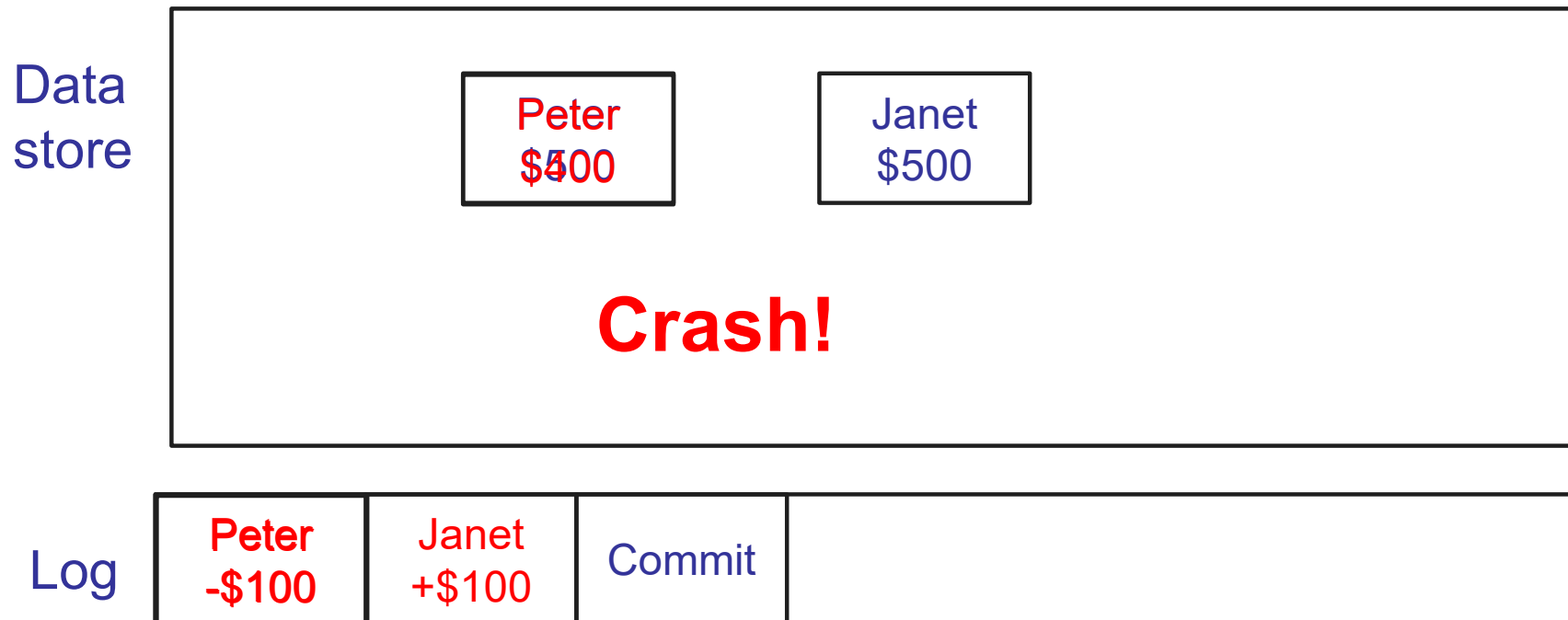


Transactions with “write-ahead” logging

- Crash before writing commit record?
 - Data store is unmodified; recovery ignores log records
- Crash after writing commit record, but before applying updates to data store?
 - Updates before commit record will be written to store during recovery
- Crash while replaying log?
 - Updates before commit record will be written to store during recovery
- Transaction committed by single sector write

Format of log records

- Why is the following log record incorrect?
- Log records must be **idempotent**: applying log record more than once is same as applying log record once

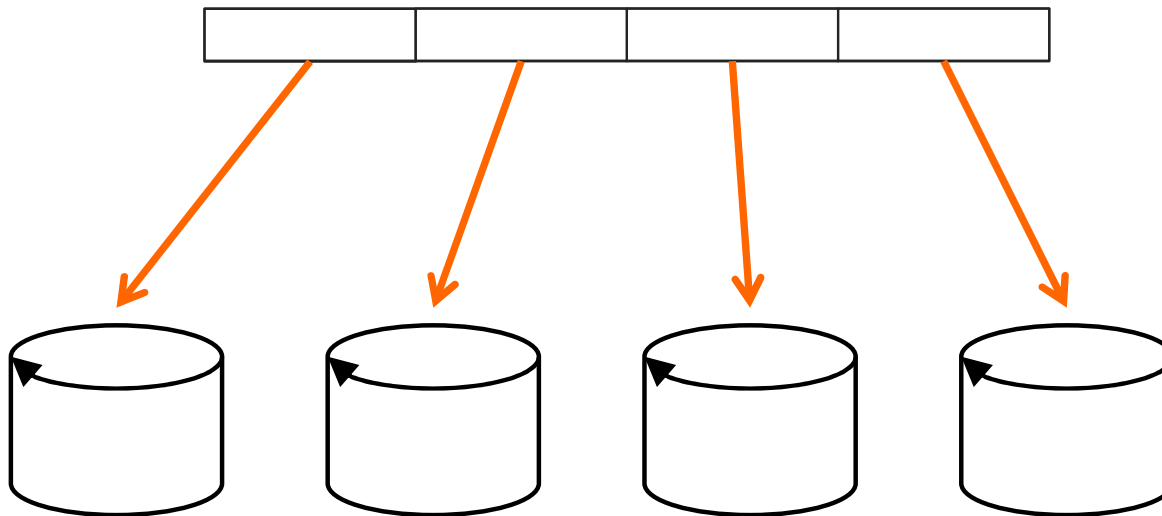


Journaling file systems

- Most modern file systems implement transactions via logging
 - Ext4, NTFS, etc.
- But usually just for file system metadata (not file data)

RAID: Redundant arrays of inexpensive disks

- Parallelize I/O across multiple disks to increase storage bandwidth and improve reliability
 - Multiple file portions are read/written in parallel
 - Bandwidth increases with more disks (storage concurrency instead of CPU concurrency)

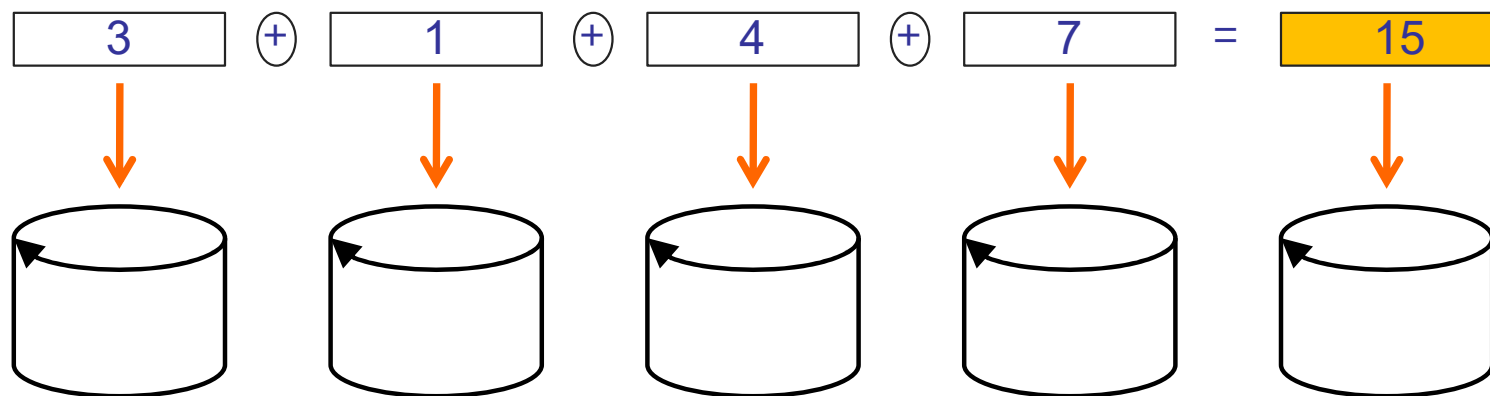


RAID reliability

- Using more disks increases probability of failure
 - E.g., if a disk has 10% probability of failing in one year, and disks fail independently, what's the probability of at least 1 of the 4 disks failing in one year?

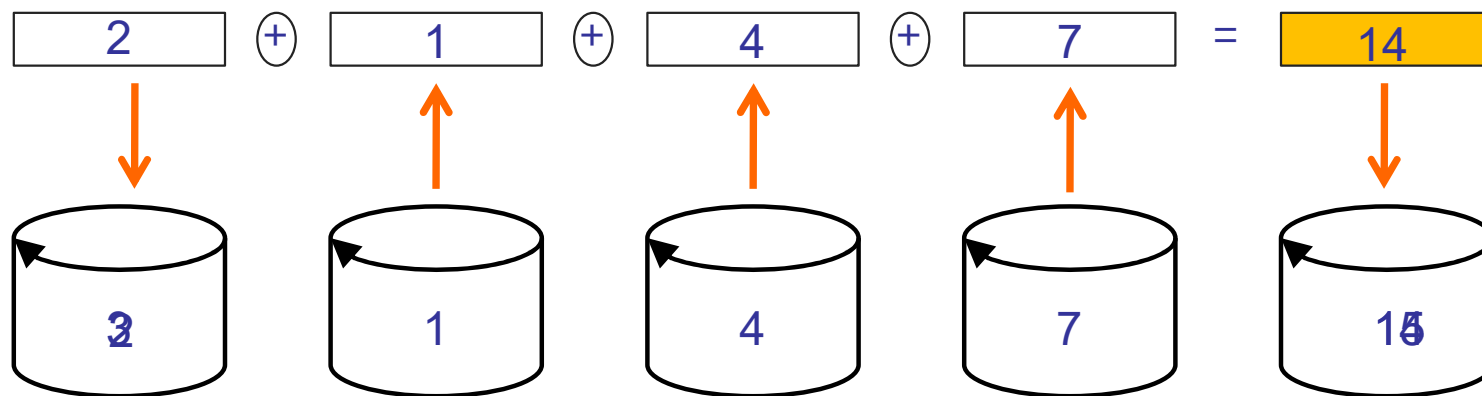
RAID with parity

- Improve reliability via **redundancy**
 - For each set of data blocks, add one **parity** block, which stores parity (XOR) of other blocks. Call this a **stripe**.
 - Add extra disk to store parity blocks.
 - Can recover any data block from all others + parity block



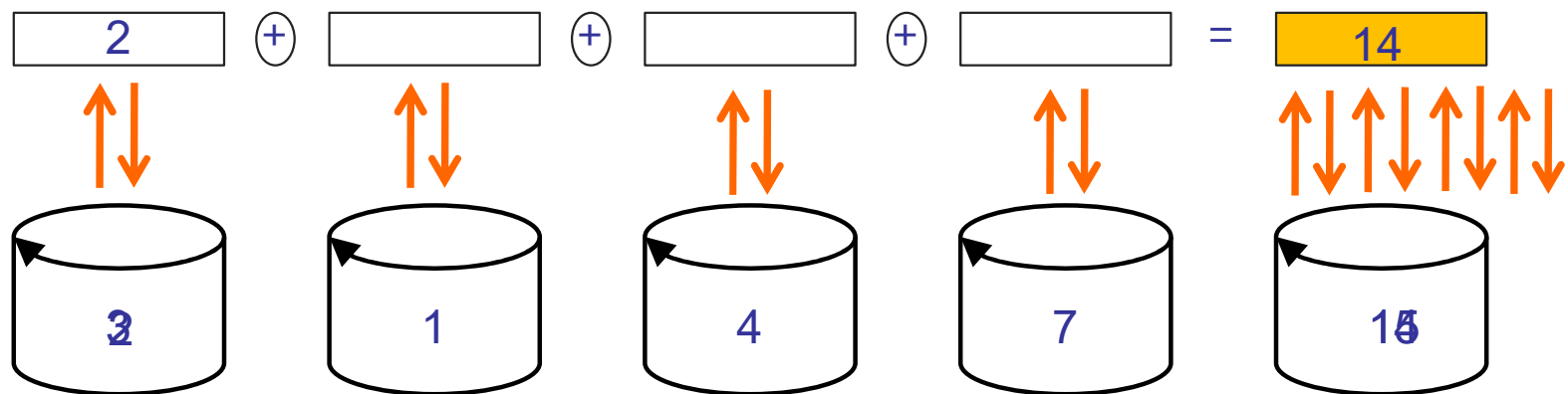
RAID with parity

- How to write a partial stripe?
 - Can we speed this up?



RAID with parity

- How to write a partial stripe?
 - Problems?



RAID levels

- **RAID 0: Striping**
 - Good performance but poor reliability
- **RAID 1: Mirroring**
 - Maintain full copy of all data
 - Good read performance, but 100% overhead for storage and writes
- **RAID 5: Floating parity**
 - Parity blocks for different stripes written to different disks
 - No single parity disk → no bottleneck at that disk